Technical Note

TN_191

FT60X Data Transfer and FIFO Master Basics

Version 1.0

Release Date: 08-07-2026

This technical note goes over FT60X data transfer basics, the Multi-Channel FIFO protocol, the 245 Synchronous FIFO protocol, and example FIFO master implementations.

Use of FTDI devices in life support and/or safety applications is entirely at the user's risk, and the user agrees to defend, indemnify, and hold FTDI harmless from any and all damages, claims, suits, or expense resulting from such use.

Future Technology Devices International Limited (FTDI)
Unit 1, 2 Seaward Place, Glasgow G41 1HH, United Kingdom
Tel.: +44 (0) 141 429 2777 Fax: + 44 (0) 141 429 2758
Web Site: http://ftdichip.com
Copyright © Future Technology Devices International Limited

Table of Contents

1 Introduction 4
2 FT60X Data Transfer Basics 5
2.1 Interfaces, Channels, and Endpoints 5
2.2 USB Host <-> FT60X Data Transfer Basics 5
3 The FT60X FIFO Protocols 9
3.1 FIFO Transaction Timing 9
3.2 MC FIFO Transaction Logic 9
3.3 245 FIFO Transaction Logic 13
4 MC FIFO Master Example 17
4.1 MC_FIFO_Controller Interface 17
4.2 MC_FIFO_Controller Operation 18
4.3 MC FIFO Master Example Module Descriptions 18
4.4 MC FIFO Master Example Constraints 20
5 245 FIFO Master Example 22
5.1 M245_FIFO_Controller Interface 22
5.2 M245_FIFO_Controller Operation 23
5.3 245 FIFO Master Example Module Descriptions 23
5.4 245 FIFO Master Example Constraints 25
6 Constraints Explanation 27
6.1 create_clock 27
6.2 set_false_path 27
6.3 set_clock_uncertainty 27
6.4 set_input_delay 27
6.5 set_output_delay 29
6.6 set_multicycle_path 30
7 Conclusion 32
8 Contact Information 33

Copyright © Future Technology Devices International Ltd

Appendix A – References 34
Document / Web References 34
Acronyms and Abbreviations 34
Appendix B – List of Tables and Figures 35
List of Tables 35
List of Figures 35
Appendix C – Revision History 37

Copyright © Future Technology Devices International Ltd

1 Introduction

A strong understanding of FT60X data transfer & FT60X FIFO master protocols is required in order to reliably use FTDI's FT60X devices. This technical note goes over FT60X data transfer basics, the Multi-Channel FIFO protocol, the 245 Synchronous FIFO protocol, and example FIFO master implementations.

Copyright © Future Technology Devices International Ltd

2 FT60X Data Transfer Basics

This section goes over FT60X data transfer basics such as the USB software interfaces, data transfer sessions with the USB host, and session underruns. Understanding these basics is critical for creating D3XX software and FT60X master implementations that interact correctly with each other.

2.1 Interfaces, Channels, and Endpoints

FTDI's FT60X devices come with two USB interfaces, a communication interface (0) and a data interface (1). The data interface can have up to 4 channels when the device is configured in Multi-Channel (MC) FIFO mode. 245 Synchronous FIFO mode uses a single channel. Each channel comes with a read and write endpoint, adding up to 8 total possible endpoints for the data interface. The communication interface comes with two endpoints, an OUT BULK endpoint for receiving Session List commands from the host, and an IN INTERRUPT endpoint for Notification List commands.

Interface	Channel	FIFO Index	Endpoint/s	Pipe Index	Description
0 Communication	N/A	N/A	0x01	0	OUT BULK endpoint for Session List commands.
			0x81	1	IN INTERRUPT endpoint for Notification List commands.
1 Data	1-4	0-3	0x02-0x05	0,2,4,6	OUT BULK endpoint for application write access. Writes data out to channels.
			0x82-0x85	1,3,5,7	IN BULK endpoint for application read access. Reads data in from channels.

Table 1 FT60X Interfaces, Channels, and Endpoints

2.2 USB Host <-> FT60X Data Transfer Basics

USB hosts will make unidirectional data read and/or write requests over the data (1) interface via software pipes assigned to individual FT60X endpoints. Each channel has its own FIFO of a specific size depending on the number of channels the FT60X is configured for as shown in Table 2. An individual FIFO master transaction for an endpoint can only transfer up to the total channel FIFO size.

Channel Configuration	FIFO Size Per Channel
1 IN	8 KiB
1 OUT	8 KiB
1	4 KiB
2	2 KiB
4	1 KiB

Table 2 FIFO Size Per Channel Configuration

Do note for all figures in this section, the FT60X indicates a channel is full/empty the moment a FIFO buffer is available to read/write and is not based on when a USB data transfer completed.

Figure 1 below is an example of a D3XX FT_ReadPipe() call where the USB host started a session on endpoint 0x82 (Channel 1 IN BULK endpoint) for 2 KiB (2048 bytes) to be transferred from the FT60X to the USB host. The FT60X in this example is in a 4-channel configuration and the USB data transfers occurring between the USB Host & FT60X are simplified. The FT60X provides the "empty" status to the FIFO master to indicate channel 1 is ready to receive data. The FIFO master in turn chooses to start a FIFO master write transaction on channel 1 and doesn't stop until the FT60X indicates channel 1's buffer is full (1 KiB / 1024 bytes). The FT60X then prepares a new 1 KiB FIFO to receive data and again provides the "empty" status to the FIFO master to indicate channel 1 is ready to receive data. The FIFO master again responds to this change in status by

Copyright © Future Technology Devices International Ltd

starting a new FIFO master write transaction for the final 1 KiB of the original 2 KiB request from the USB host.

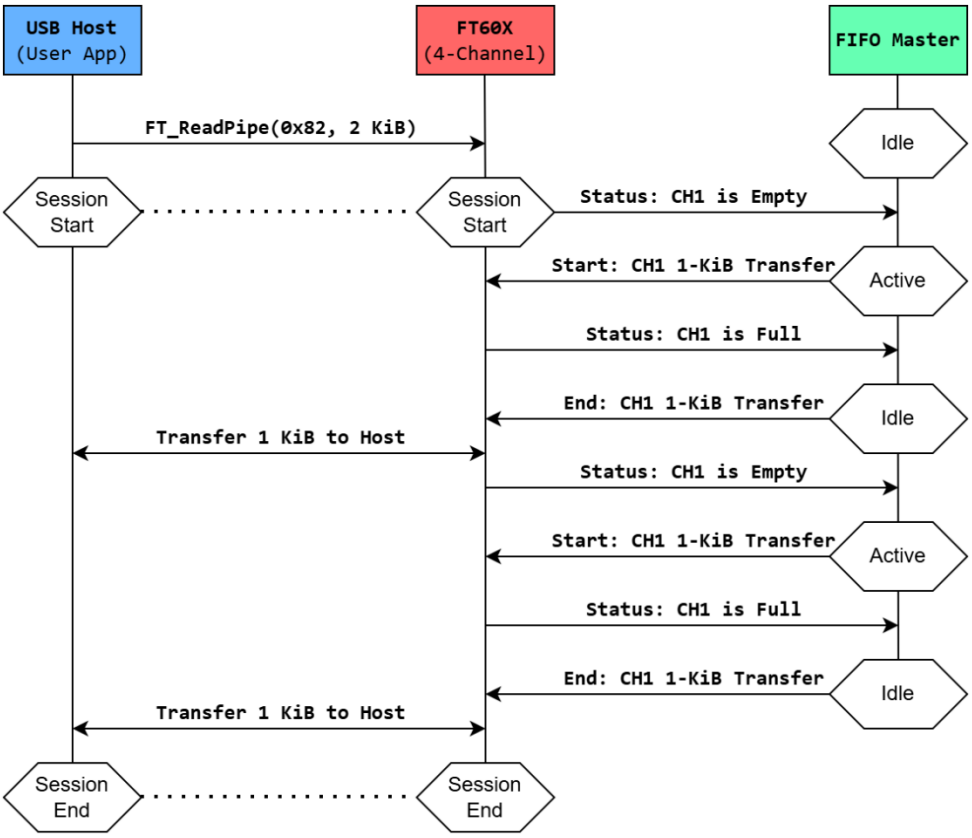


Figure 1 FT60X 4-Channel Normal Session

During the whole read pipe session in Figure 1 between the USB Host & FT60X, the example FIFO master has no knowledge of how large the FT_ReadPipe() call was, nor how large the FIFO size is for each channel. It simply started write transactions when possible and sent data until the FT60X indicated it was full. If in Figure 1 the host originally made a FT_ReadPipe call for 1028 bytes, the session would look the same except the last FIFO transaction would be 4 bytes. Figure 1 is an example of a normal read session between the host & FT60X.

Figure 2 below is an example of a session underrun, where the FIFO master ended a FIFO transaction prematurely before the entire request for 2 KiB was fulfilled. In this scenario the session ends without the user getting all requested bytes and the FT60X will not indicate CH1 is empty again until a new FT_ReadPipe() call is made to start a new session.

Copyright © Future Technology Devices International Ltd

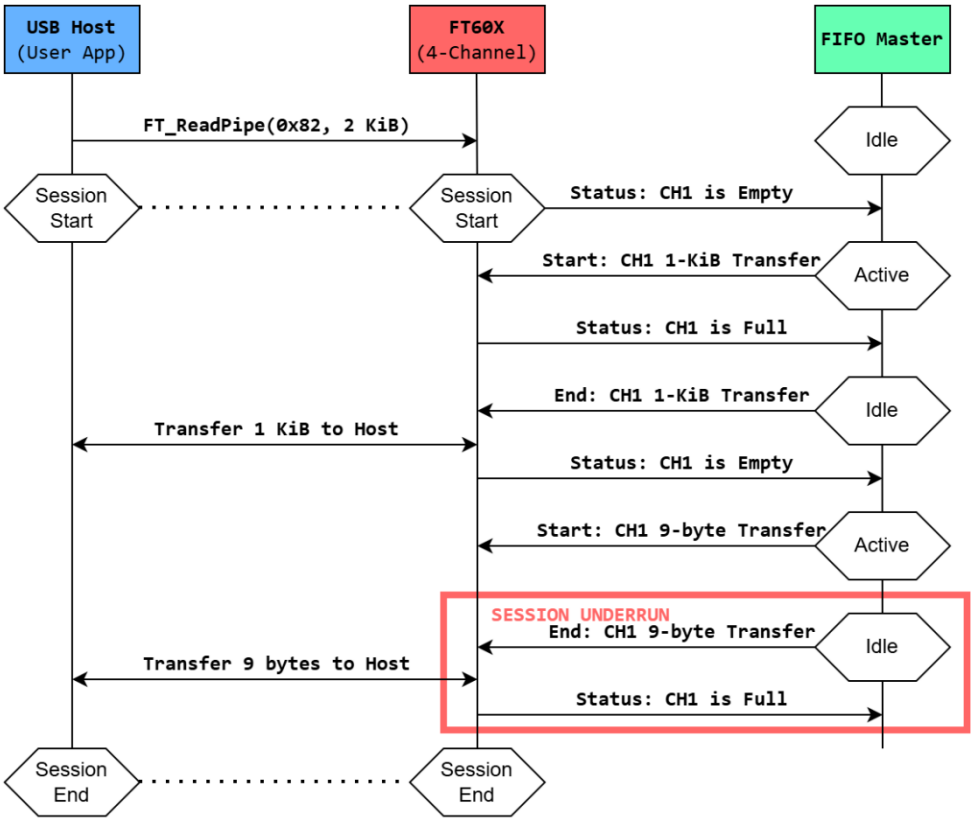


Figure 2 FT60X 4-Channel Session Underrun

Figure 3 below is an example of an FT_ReadPipe() session for 16 bytes, where a FT601 is configured to ignore session underruns on multiples of FIFO bus-width bytes. Using the FT601 which has a bus-width of 4 bytes as an example, this means as long as each FIFO master transaction is divisible by 4 bytes the session will not end until the request for all 16 bytes is fulfilled. For the FT600 which has a bus-width of 2 bytes, the same logic applies where each FIFO master transaction must be divisible by 2 bytes.

Copyright © Future Technology Devices International Ltd

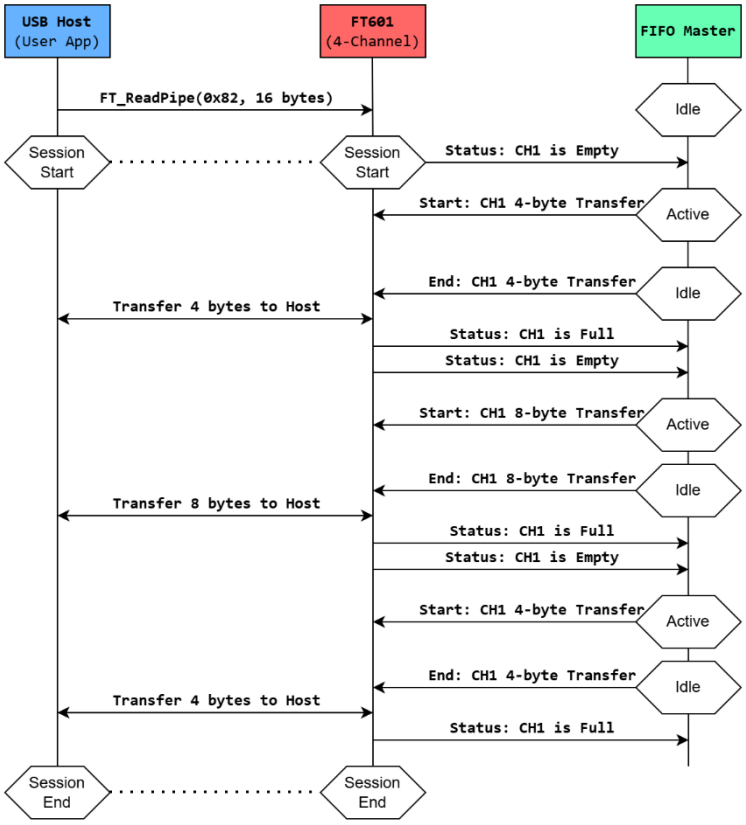


Figure 3 FT601 4-Channel Ignore Bus-Width Underruns Session

In Figure 3, if the FT_ReadPipe() call was for 15 bytes instead, the session would look exactly the same except the FT601 would accept 3 bytes from the last FIFO transaction. Notice in Figure 3 the 8-byte FIFO master transaction is shown to indicate transactions can be any size divisible by the FIFO bus-width. The FIFO master could have fulfilled this example session with four 4-byte transactions, two 8-byte transactions, or a single 16-byte transaction instead.

The FT60X can also be configured to instead ignore session underruns on multiples of MaxPacketSize bytes where MaxPacketSize is determined by the current USB speed the FT60X is operating at. Table 3 below shows the MaxPacketSize for all possible USB speeds. The same logic applies where a session will not end as long as all FIFO master transactions are divisible by the MaxPacketSize (as opposed to the FIFO bus-width).

USB Speed	MaxPacketSize
USB 3.0 SuperSpeed	1024 Bytes
USB 2.0 High Speed	512 Bytes
USB 2.0 Full Speed	64 Bytes

Table 3 MaxPacketSize Values

A D3XX FT_WritePipe() call is a write request to transfer data from the host to the FIFO master. There is no equivalent session structure like for read requests (FT_ReadPipe()), the FIFO master is expected to read out all data possible over multiple full FIFO master read transactions as needed. Otherwise, it is possible for a bottleneck to form and write requests from the host to time out.

Copyright © Future Technology Devices International Ltd

3 The FT60X FIFO Protocols

This section goes over the logic & timing of the Multi-Channel (MC), & 245 Synchronous (245) FIFO protocols. The FT60X is always the FIFO slave and never initiates a FIFO transaction. The FIFO master is in full control of when FIFO transactions start and end. However, the FIFO master must always keep the state of the FIFO slave (FT60X) in mind before, during, and after transactions in order to avoid the FIFO slave (FT60X) from entering a bad state.

The FIFO master should only operate off the positive clock edge of the clock provided by the FIFO slave (FT60X), and always guarantee its output is valid around the subsequent positive clock edge. The FT60X datasheet and Section 3.1 go over the minimum required valid setup & hold times around the positive edge of the clock.

3.1 FIFO Transaction Timing

Both the FIFO master and slave (FT60X) expect valid input around the positive edge of the clock provided by the FIFO slave. They in turn will provide valid output around the subsequent positive clock edge. Figure 4 and Table 4 show the minimum valid setup and hold times for output.

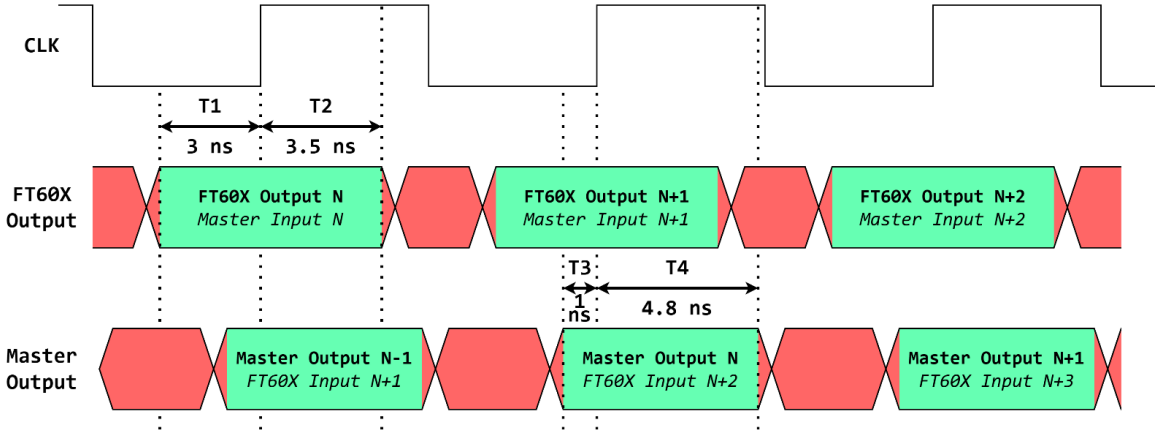


Figure 4 FIFO Bus Input/Output Timing Diagram

Time	Description	Minimum Time Length
T1	Slave Drive Output Set Up Time	3 ns
T2	Slave Drive Output Hold Time	3.5 ns
T3	Master Drive Output Set Up Time	1 ns
T4	Master Drive Output Hold Time	4.8 ns

Table 4 FIFO Bus Output Timing

3.2 MC FIFO Transaction Logic

The MC FIFO protocol consists of five phases, IDLE (IDL), Command (CMD), Bus turn around 1 (TR1), Data (DAT), and Bus turn around 2 (TR2). The master can initiate either a write or read transaction when appropriate, and transactions consist of all phases except the IDLE phase. Table 5 shows the state of the MC FIFO bus in all phases. For the FT600, you can ignore DATA[31:16] & BE[3:2] as they don't exist for it. **The TXE_N signal is ignored in this section as it is the only optional signal.** You can refer to the FT60X datasheet for more information about the TXE_N signal.

Copyright © Future Technology Devices International Ltd

MC FIFO Bus Lines	IDLE (IDL)	Command (CMD)	Bus turn around 1 (TR1)		Data (DAT)		Bus turn around 2 (TR2)	
WR_N	1'b1	1'b0	1'b0		1'b0		1'b1*	
RXF_N	1'b1	1'b1	1'b1		1'b0		1'b1*	
DATA[31:16]	16'hFFFF	16'hFFFF	Write 16'hFFFF	Read 16'hXXXX	Write Data	Read Data	16'hFFFF	
DATA[15:8]	FIFO Status	FIFO Status	8'hXX		Data	Data	8'hXX	
DATA[7:0]	8'hFF	Channel	Write Channel	Read 8'hXX	Data	Data	8'hFF	
BE[3:0]	4'hF	Command	Write 4'h1	Read 4'X	BE	BE	Write 4'hF	Read 4'X

= FIFO Slave (FT60X) is driving the line/s.
= FIFO Master is driving the line/s.
= FIFO Master should ignore the line/s.

Table 5 MC FIFO Bus Phases

WR_N is the FIFO bus master transaction request signal. The bus master pulls this signal low (1'b0) during a bus transaction.

RXF_N is the FIFO bus slave data acknowledge signal. This signal is only low (1'b0) during data (DAT) phase clock cycles where the data was successfully received by or validly sent from the FIFO slave.

In Table 5, a phase column is split in two if the behaviour of the lines differentiates based off "Command" being for a Write (BE = 4'h1) or a Read (BE = 4'h0) transaction. Table 6 below shows the possible values for "Channel" & "Command" present in the Command phase.

FIFO Transaction Type	BE[3:0] Command	DATA[7:0] Channel
Read	4'h0	8'h1 = Channel 1 8'h2 = Channel 2 8'h3 = Channel 3 8'h4 = Channel 4
Write	4'h1	

Table 6 MC Command Phase Values

Table 7 below shows how the "FIFO Status" in the IDLE (IDL) phase for each endpoint is indicated by the FIFO slave (FT60X).

FIFO Status	Channel	Endpoint	Value
DATA[15]	4	0x05	1'b0 = CH4 ready for FIFO read transaction. 1'b1 = CH4 not ready for FIFO read transaction.
DATA[14]	3	0x04	1'b0 = CH3 ready for FIFO read transaction. 1'b1 = CH3 not ready for FIFO read transaction.
DATA[13]	2	0x03	1'b0 = CH2 ready for FIFO read transaction. 1'b1 = CH2 not ready for FIFO read transaction.
DATA[12]	1	0x02	1'b0 = CH1 ready for FIFO read transaction. 1'b1 = CH1 not ready for FIFO read transaction.
DATA[11]	4	0x85	1'b0 = CH4 ready for FIFO write transaction. 1'b1 = CH4 not ready for FIFO write transaction.
DATA[10]	3	0x84	1'b0 = CH3 ready for FIFO write transaction. 1'b1 = CH3 not ready for FIFO write transaction.
DATA[9]	2	0x83	1'b0 = CH2 ready for FIFO write transaction. 1'b1 = CH2 not ready for FIFO write transaction.
DATA[8]	1	0x82	1'b0 = CH1 ready for FIFO write transaction. 1'b1 = CH1 not ready for FIFO write transaction.

Table 7 MC FIFO Status Bits

Copyright © Future Technology Devices International Ltd

Referring to Table 7, DATA[14] = 1'b0 means there was a write request (FT_WritePipe()) from the host to the FT60X on channel CH3 (endpoint 0x04), and now data is available to be read from channel 3 via FIFO master read transactions.

Again referring to Table 7, DATA[9] = 1'b0 means there was a read request (FT_ReadPipe()) from the host to the FT60X on channel 2 (endpoint 0x83), and now the FIFO master is able to write data to channel 2 via FIFO master write transactions.

Table 8 below shows possible values for "BE" (Byte Enable) during the Data (DAT) phase. BE must be 4'b1111 for all clock cycles during the Data phase except for the last clock cycle. BE can be any of the possible values in Table 8 during the last clock cycle of the Data phase. If BE is not 4'b1111 that indicates to the FIFO slave/master, its master/slave has no more data available for the next clock cycle and the transaction is essentially over. The FIFO master drives BE[3:0] during FIFO write transactions, and the FIFO slave drives BE[3:0] during FIFO read transactions. The FT600 can ignore BE[3:2] as they don't exist for it.

BE[3:0] Value	Description
4'b1111	Data[31:0] (4 bytes) is valid. May or may not be last cycle of Data phase.
4'b0111	Data [23:0] (3 bytes) is valid. Last cycle of Data phase.
4'b0011	Data[15:0] (2 bytes) is valid. FT601: Last cycle of Data phase. FT600: May or may not be last cycle of Data phase.
4'b0001	Data[7:0] (1 bytes) is valid. Last cycle of Data phase.
Other	Invalid, BE[3:0] must be one of the above values. Otherwise you're in a bad state.

Table 8 Data Phase Byte Enable Values

All logical waveform diagrams in Section 3.2 specifically do not represent accurate setup and hold times. Green and Red represent periods with settled output values. Green represents acknowledged values and Red represents ignored values. Input should never be clocked in on the negative clock edge as setup and hold times only guarantee settled valid output around the positive edges.

The below logical waveform diagram Figure 5 shows a full FIFO master read transaction on channel 4 where the FIFO slave only had 11 bytes available to send for the master to receive.

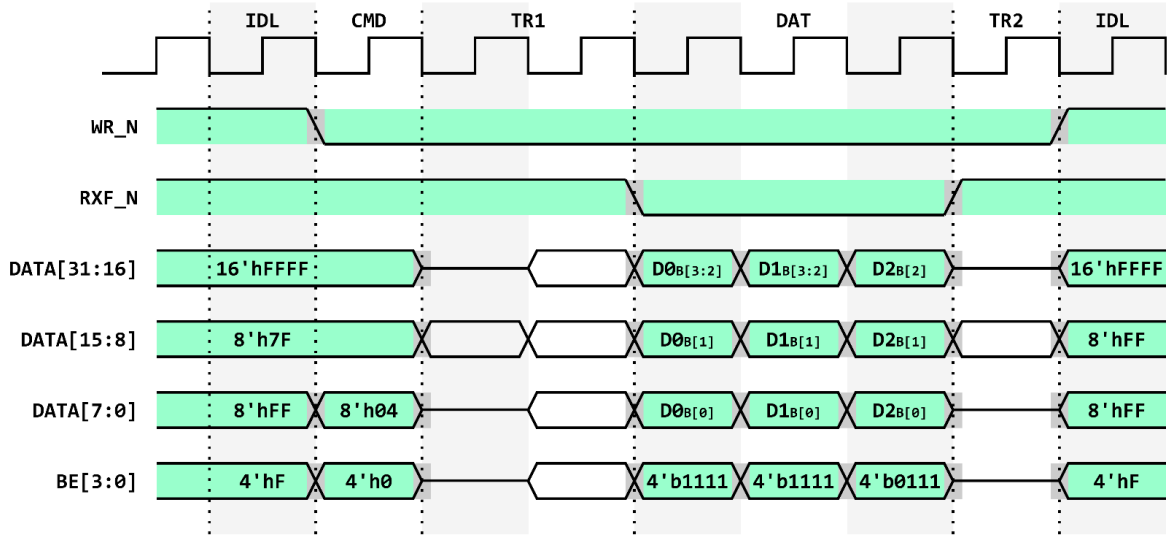


Figure 5 MC FIFO Master Full Read Transaction

Copyright © Future Technology Devices International Ltd

Note in Figure 5, the last 3 bytes (**$D2_{B[2:0]}$**) of the 11-byte data transaction are present in the last data (DAT) phase clock cycle with BE[3:0] = 4'b0111. BE[3] = 1'b0 indicates the fourth byte in that clock cycle (**$D2_{B[3]}$**) was invalid/not enabled.

The below logical waveform diagram Figure 6 shows a partial FIFO master read transaction on channel 3, where the FIFO master only had 8 bytes of space left to receive data and ended the transaction early before the entire slave's FIFO buffer was read.

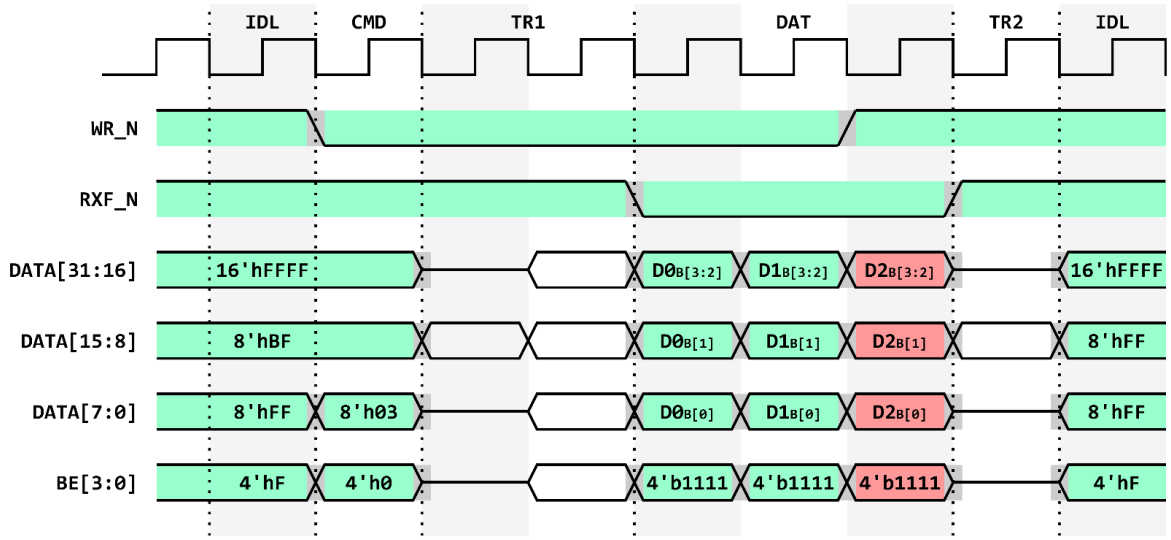


Figure 6 MC FIFO Master Partial Read Transaction

Note in Figure 6, the FIFO slave still sent **$D2_{B[3:0]}$** on the bus. However **$D2_{B[3:0]}$** will still remain in the FIFO slave buffer as the master did not acknowledge receiving **$D2_{B[3:0]}$** that clock cycle (WR_N was high).

The below logical waveform diagram Figure 7 shows a partial FIFO master write transaction on channel 2, where the FIFO master only had 13 bytes of data to send to the slave which was expecting more.

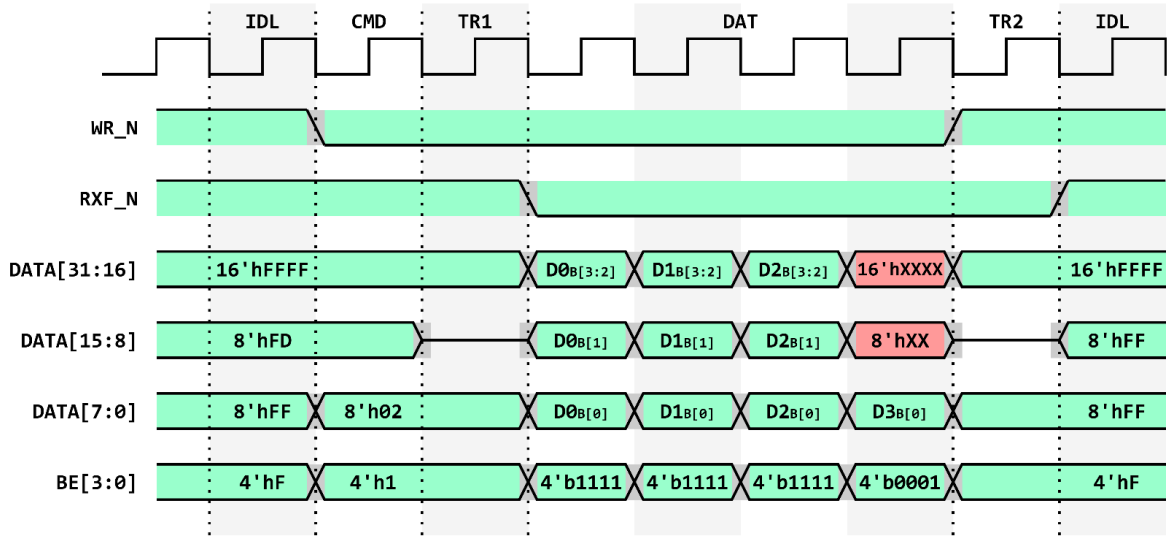


Figure 7 MC FIFO Master Partial Write Transaction

Note in Figure 7 only the first byte of **D3** was enabled and transferred to the FIFO slave because the FIFO master drove BE[3:0] as 4'b0001.

The below logical waveform diagram Figure 8 shows a full FIFO master write transaction on

Copyright © Future Technology Devices International Ltd

channel 2, where the FIFO slave buffer was filled & the FIFO master by chance had more data to send.

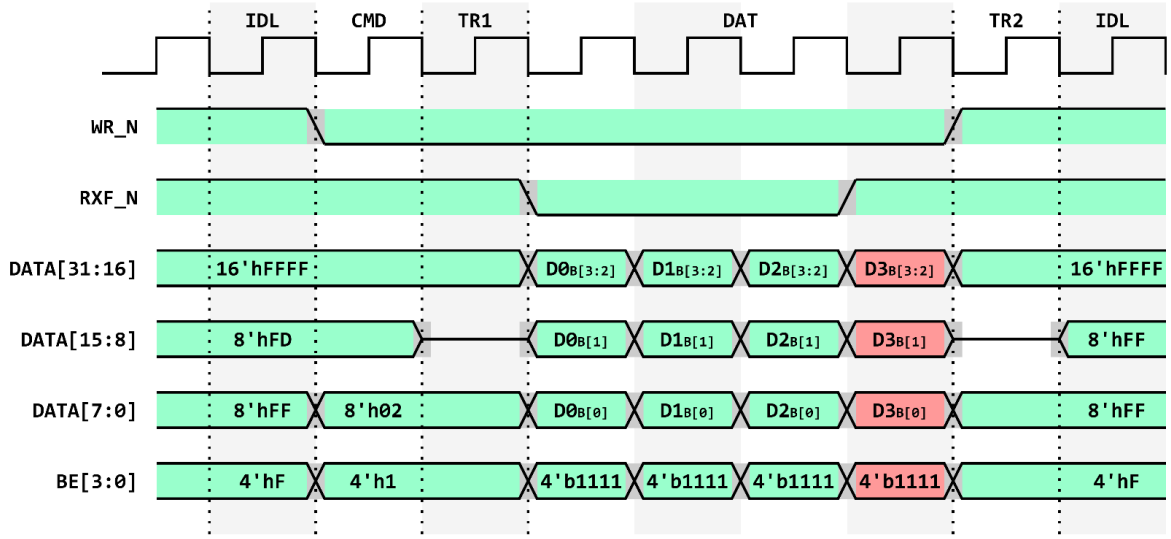


Figure 8 MC FIFO Master Full Write Transaction

Note in Figure 8, the FIFO master had an extra 4 bytes (**D3**) to send, but RXF_N was 1'b1. In this scenario the FIFO master knows the FIFO slave did not receive those bytes (**D3**).

3.3 245 FIFO Transaction Logic

The FT60X 245 Synchronous FIFO protocol does not explicitly consist of phases but will be split into three phases to more easily explain/show the transaction logic. We split the 245 FIFO protocol into three phases, Idle (IDL), Output Transition Phase (OTP), & Data (DAT). The master can initiate either a write or read transaction when appropriate, and transactions can consist of all phases except the Idle phase. Table 9 shows the state of the 245 FIFO bus in all phases. For the FT600, you can ignore DATA[31:16] & BE[3:2] as they don't exist for it.

245 FIFO Bus Lines	Idle (IDL)	Output Transition Phase (OTP)	Data (DAT)	
TXE_N	1'bX	1'bX	Write	Read
			1'b0	1'bX
RXF_N	1'bX	1'b0	1'bX	1'b0
OE_N	1'b1	1'b0	1'b1	1'b0
RD_N	1'b1	1'b1	1'b1	1'b0
WR_N	1'b1	1'b1	1'b0	1'b1
DATA[31:0]	32'hXX	32'hXX	Data	Data
BE[3:0]	4'hX	4'hX	BE	BE

= FIFO Slave (FT60X) is driving the line/s.
= FIFO Master is driving the line/s.
= FIFO Master should ignore the line/s.

Table 9 245 FIFO Bus Phases

In Table 9, a phase column is split in two if the behaviour of the lines differentiates based off the bus undergoing a FIFO master Write transaction or a FIFO master Read transaction.

TXE_N is the FIFO slave Transmit FIFO Empty signal. When the slave pulls this signal low, the FIFO slave has space to receive data sent by the master.

RXF_N is the FIFO slave Receive FIFO Full signal. When the slave pulls this signal low, the FIFO slave has data that can be read by the master.

Copyright © Future Technology Devices International Ltd

OE_N is the FIFO master Output Enable signal. When the master pulls this signal low, the FIFO slave will drive DATA[31:0] & BE[3:0] on the subsequent clock cycle.

RD_N is the FIFO master Read Enable signal. It is driven low by the master during data phase clock cycles where the master successfully received data from the slave. You will violate the protocol and not save a clock cycle by initially pulling RD_N & OE_N low at the exact same time.

WR_N is the FIFO master Write Enable signal. It is driven low by the master during data phase clock cycles where the master has valid data on the bus for the slave to read.

Table 8 from earlier shows possible values for "BE" (Byte Enable) during the Data (DAT) phase. BE must be 4'b1111 for all clock cycles during the Data phase except for the last clock cycle. BE can be any of the possible values in Table 8 during the last clock cycle of the Data phase. If BE is not 4'b1111 that indicates to the FIFO slave/master, its master/slave has no more data available for the next clock cycle and the transaction is essentially over. The FIFO master drives BE[3:0] during FIFO write transactions, and the FIFO slave drives BE[3:0] during FIFO read transactions. The FT600 can ignore BE[3:2] as they don't exist for it.

All logical waveform diagrams in Section 3.3 specifically do not represent accurate setup and hold times. Green and red represent periods with settled output values. Green represents acknowledged values and Red represents ignored values. Input should never be clocked in on the negative clock edge as setup and hold times only guarantee settled valid output around the positive edges.

The below logical waveform diagram Figure 9 shows a full FIFO master read transaction where the FIFO slave only had 15 bytes available to send for the master to receive.

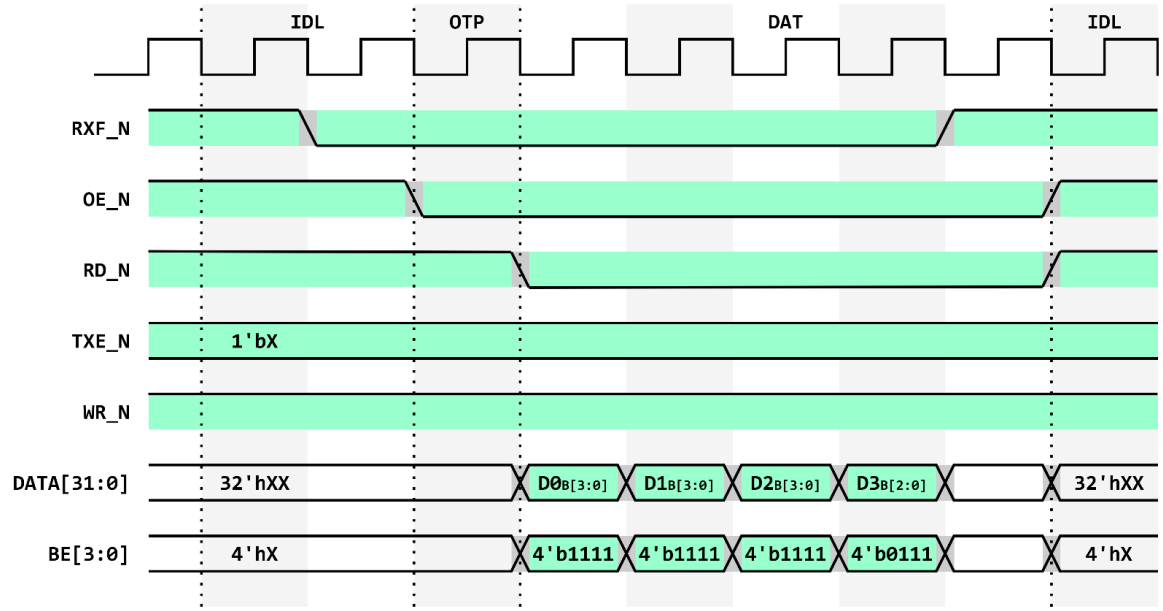


Figure 9 245 FIFO Master Full Read Transaction

Note in Figure 9, the last 3 bytes (**$D3_{B[2:0]}$**) of the 15-byte data transaction are present in the last data (DAT) phase clock cycle with BE[3:0] = 4'b0111 & RXF_N = 1'b0. BE[3] = 1'b0 indicates the fourth byte in that clock cycle (**$D3_{B[3]}$**) was invalid/not enabled.

The below logical waveform diagram Figure 10 shows a partial FIFO master read transaction where the FIFO master only had 16 bytes of space left to receive data and ended the transaction early before the entire slave's FIFO buffer was read.

Copyright © Future Technology Devices International Ltd

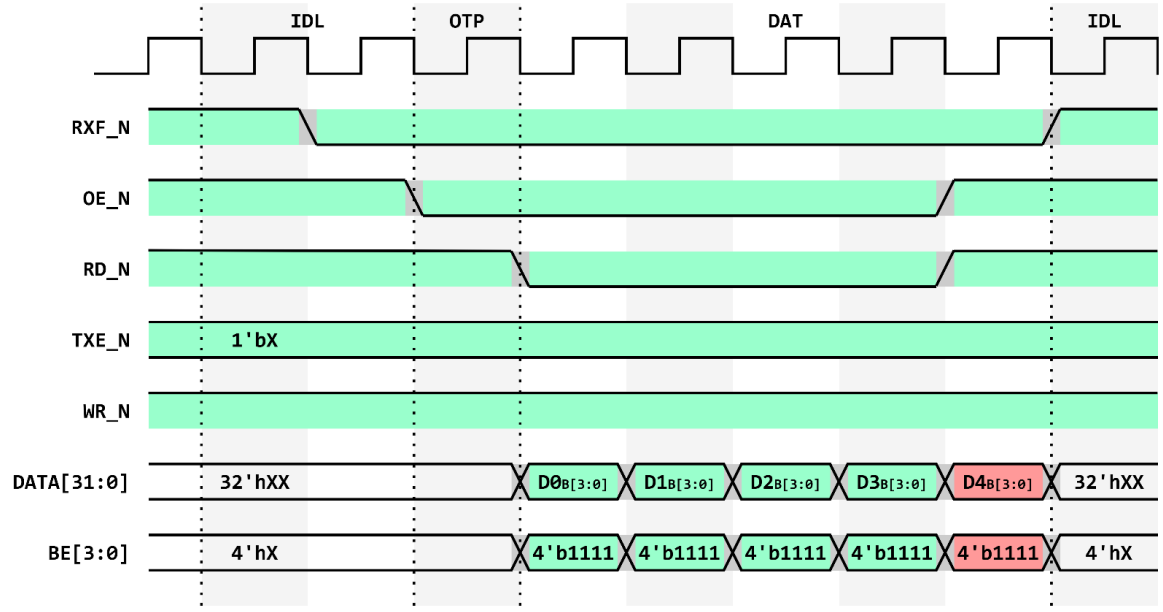


Figure 10 245 FIFO Master Partial Read Transaction

Note in Figure 10, the FIFO slave still sent **$D4_{B[3:0]}$** on the bus. However **$D4_{B[3:0]}$** will still remain in the FIFO slave buffer as the master did not acknowledge receiving **$D4_{B[3:0]}$** that clock cycle (RD_N was high).

The below logical waveform diagram Figure 11 shows a partial FIFO master write transaction where the FIFO master only had 17 bytes of data to send to the slave which was expecting more.

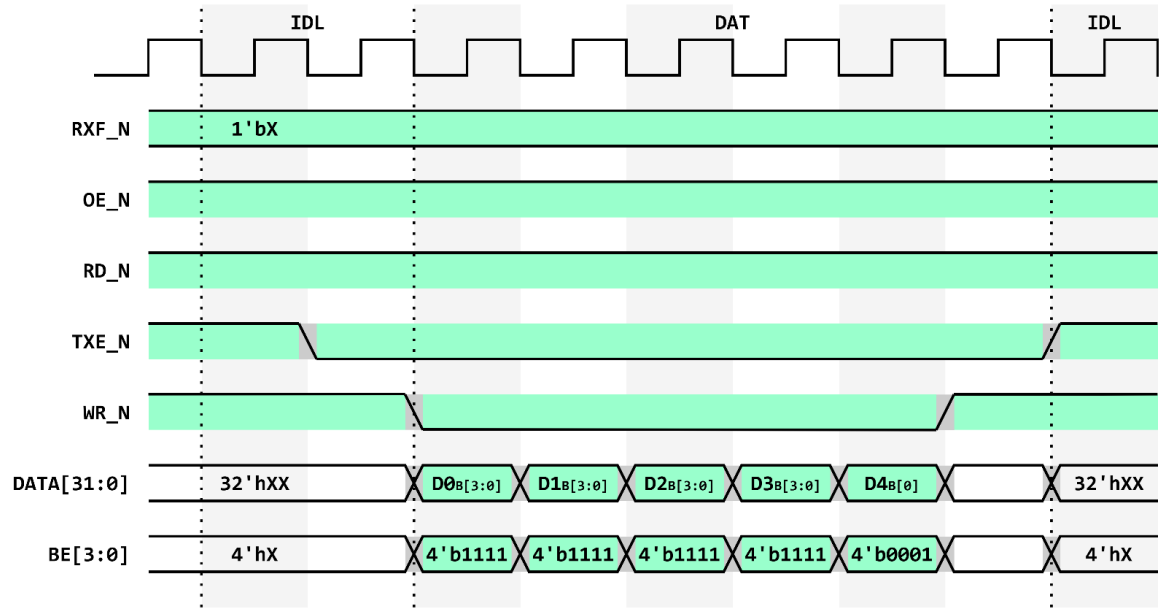


Figure 11 245 FIFO Master Partial Write Transaction

Note in Figure 11 only the first byte of D4 was enabled and transferred to the FIFO slave because the FIFO master drove BE[3:0] as 4'b0001.

The below logical waveform diagram Figure 12 shows a full FIFO master write transaction where the FIFO slave buffer was filled & the FIFO master by chance had more data to send.

Copyright © Future Technology Devices International Ltd

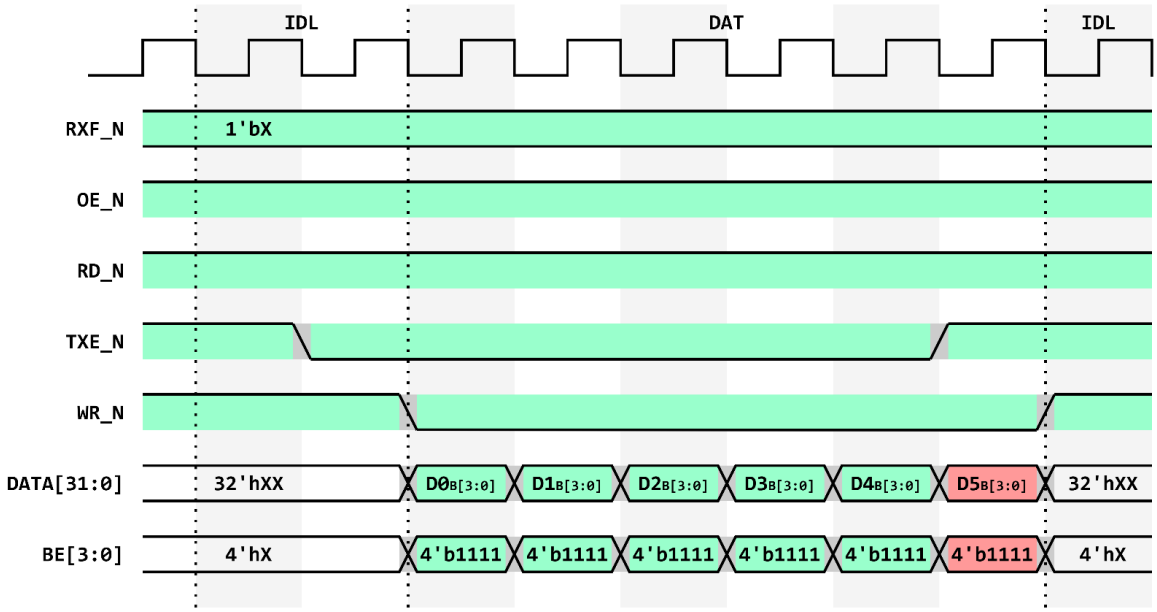


Figure 12 245 FIFO Master Full Write Transaction

Note in Figure 12, the FIFO master had an extra 4 bytes (**D5**) to send, but TXE_N was 1'b1. In this scenario the FIFO master knows the FIFO slave did not receive those bytes (**D5**).

Copyright © Future Technology Devices International Ltd

4 MC FIFO Master Example

This section goes over a Multi-Channel FIFO master example. The Quartus project for this example can be downloaded at: FT60X Examples

4.1 MC_FIFO_Controller Interface

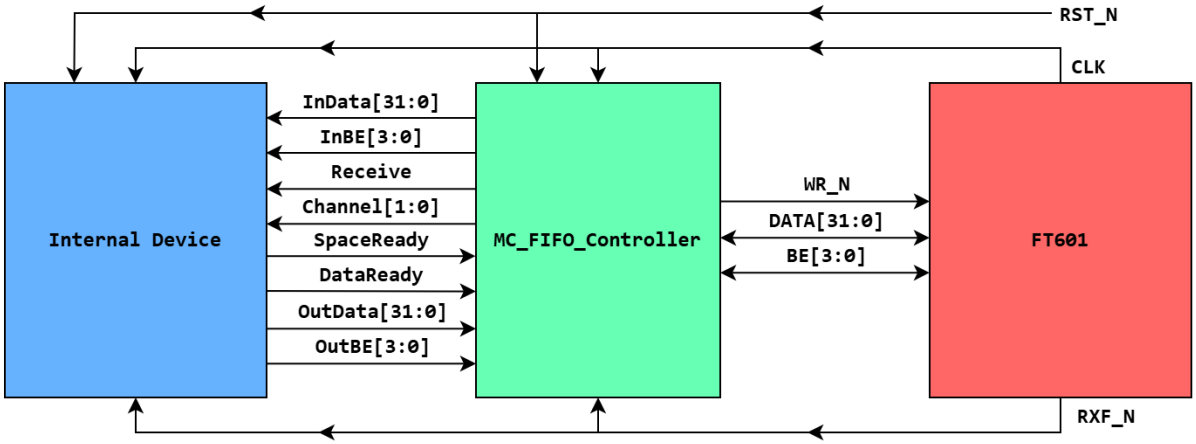


Figure 13 MC_FIFO_Controller Interface Diagram

Figure 13 shows how our example FIFO master (MC_FIFO_Controller) is connected to a FT601 and an Internal Device (some arbitrary module) that needs to send/receive data to/from the FT601. The MC_FIFO Controller in this example is essentially another data bridge. Table 10 goes over the interface between the MC_FIFO_Controller & the Internal Device. Only RXF_N from the FT601 is directly passed to the Internal Device.

Internal Device Interface	Internal Device Direction	Description
InData[31:0]	Input	Data for Internal Device to receive.
InBE[3:0]	Input	BE for Internal Device to receive.
Receive	Input	1'b1 = Internal Device must store incoming data. 1'b0 = Internal Device must send incoming data.
Channel[1:0]	Input	1'b00 = Channel 1 needs to send/receive data. 1'b01 = Channel 2 needs to send/receive data. 1'b10 = Channel 3 needs to send/receive data. 1'b11 = Channel 4 needs to send/receive data.
SpaceReady	Output	Internal Device is ready to receive data.
DataReady	Output	Internal Device is ready to send data.
OutData[31:0]	Output	Data to send to FT601.
OutBE[3:0]	Output	BE to send to FT601.
* RXF_N *	Input	From FT601 directly to Internal Device. 1'b0 = Acknowledge successful data read/write cycle. 1'b1 = No data transfer.

Table 10 MC Internal Device Interface

The MC_FIFO_Controller handles the MC FIFO protocol and scheduling transactions on individual channels. The Internal Device only needs to present its ability to send/receive (SpaceReady/DataReady) on the specific channel given by MC_FIFO_Controller. If the Internal Device is able to send/receive on said channel, the MC_FIFO_Controller will initiate a FIFO transaction. If the Internal Device was by chance a FIFO buffer, it would only increment/decrement its internal FIFO address on clock cycles where RXF_N was low.

Copyright © Future Technology Devices International Ltd

4.2 MC_FIFO_Controller Operation

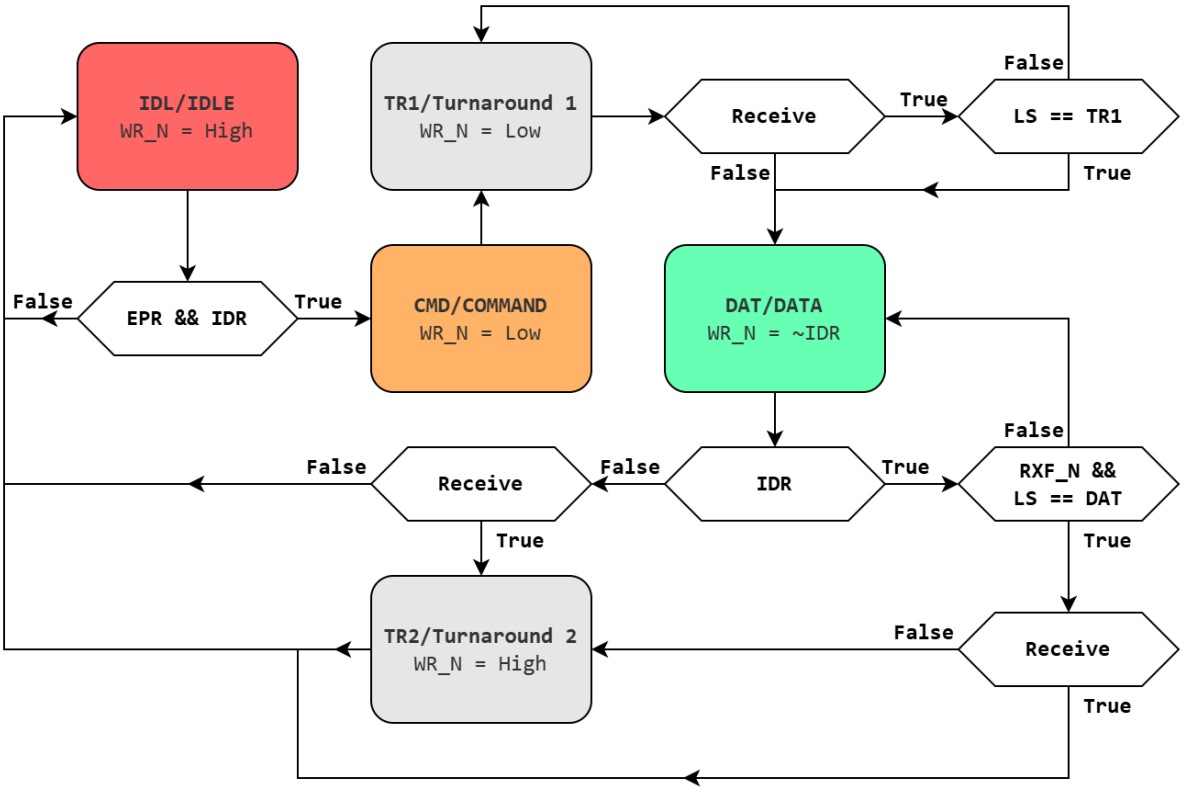


Figure 14 MC_FIFO_Controller State Machine

Figure 14 shows the state machine of the MC_FIFO_Controller. The state machine only has five states (IDL, CMD, TR1, DAT, and TR2), logic block paths between states are evaluated in one transition. Table 11 describes all the unique values used in the state machine. When in the IDL state, the MC_FIFO_Controller checks if a specific endpoint is ready to send/receive data and changes which endpoint it checks each clock cycle.

State Machine Value	Description
EPR (Endpoint Ready)	Is true if the FT601 endpoint is ready to send/receive data.
IDR (Internal Device Ready)	Is true if the Internal Device is ready to send/receive data.
Receive	If true, the Internal Device must receive data from the FT601. If false, the Internal Device must send data to the FT601.
LS (Last State)	What the previous state of the MC_FIFO_Controller was.

Table 11 MC_FIFO_Controller State Machine Unique Values

4.3 MC FIFO Master Example Module Descriptions

This section goes over the example MC FIFO Master module descriptions.

Copyright © Future Technology Devices International Ltd

Module	Description
MC_FIFO_Master.sv	The top-level module that interfaces between the FT601 and the MC_FIFO_Controller. Is purely for interfacing and has no logic.
MC_FIFO_Controller.sv	The MC_FIFO_Controller module that acts as a MC FIFO protocol bridge between the FT601 and some Internal Device (module). It uses a round-robin method to check which endpoints are ready to send/receive data and will initiate transactions if the Internal device is also ready to send/receive data.
MC_FIFO_Buffer.sv	An example Internal Device module. It is a data loopback FIFO buffer with 8 KiB of space per channel (32 KiB total). It will store data sent by the FT601 and send it back to the FT601 when requested.
BYTE_RAM.sv	1-byte x 8192 word RAM IP block (BRX). MC_FIFO_Buffer uses four of these to retrieve/store values transferred on the bus. MC_FIFO_Buffer uses logic and these individual buffers so it can store/read memory at 1-byte boundaries as opposed to 4-byte boundaries. This allows MC_FIFO_Buffer to store 8191 bytes from one write pipe call and still have 1 byte of space left that can store a byte from another write pipe call. Ensuring space is never wasted in the RAM.
MC_FIFO_Counter.sv	An example Internal Device module. It is a 4-channel 32-bit counter that is always ready to send/receive data. When receiving data (RXF_N = 1'b0), the 32-bit counter will be set to the last cycle of data received. When sending data (RXF_N = 1'b0), the 32-bit counter will send its current value and increment every clock cycle. If the FT601 requested 16 bytes and the counter was set to 0. The FT601 would receive four 32-bit values (0, 1, 2, & 3). If the FT601 wrote 16 bytes, the counter would be set to the value of the last four bytes. If the FT601 wrote 32'd5, and then requested 4 bytes of data, it would receive 32'd5 back and the counter would increment to 32'd6.
MC_FIFO_Video.sv	An example Internal Device module. It is a 4-channel YUV422 video data generator. It sends 1920x1080 YUV422 frames of a color-changing bouncing box to the USB host. A FT601 will receive the raw header and video data directly. A FT602 will display actual video.

Table 12 MC Master Example Module Descriptions

The below block (Figure 15) in MC_FIFO_Controller.sv can be modified to change what Internal Device example module is attached to the MC_FIFO_Controller (change MC_FIFO_Buffer to MC_FIFO_Counter and vice-versa). You can also modify it to use your own custom module.

Copyright © Future Technology Devices International Ltd

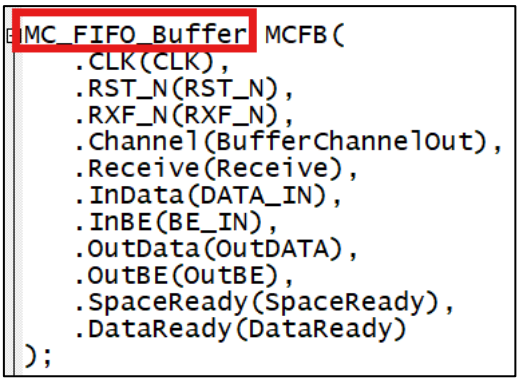


Figure 15 MC_FIFO_Controller.sv Internal Device Block

4.4 MC FIFO Master Example Constraints

Below are the constraints used for the MC FIFO master example. The constraints are gone into more depth in Section 6.0.

```
#Specify the clock period.
set Period 10.000
set InputSetupTime 3.0
set InputHoldTime 3.5
set OutputSetupTime 1.0
set OutputHoldTime 4.8

create_clock -name C_CLK -period $Period [get_ports {CLK_FIFO}]

# Don't care about RST_N timing. Non-critical, Set false path.
set_false_path -from [get_ports {RST_N}] -to [get_registers *]

# Set a high clock uncertainty of 10% to increase reliability.
set_clock_uncertainty [expr $Period / 10.0] -from [get_clocks C_CLK] -to
[get_clocks C_CLK]

#Input constraints are correct.
set_input_delay -add_delay -clock [get_clocks C_CLK] -max [expr 0 -
$InputSetupTime] [get_ports {RXF_N}]
set_input_delay -add_delay -clock [get_clocks C_CLK] -max [expr 0 -
$InputSetupTime] [get_ports {BE[*] DATA[*]}]
set_input_delay -add_delay -clock [get_clocks C_CLK] -min [expr 0 +
$InputHoldTime] [get_ports {RXF_N}]
set_input_delay -add_delay -clock [get_clocks C_CLK] -min [expr 0 +
$InputHoldTime] [get_ports {BE[*] DATA[*]}]

#Output constraints are correct.
set_output_delay -add_delay -clock [get_clocks C_CLK] -max [expr
$OutputSetupTime] [get_ports {WR_N}]
set_output_delay -add_delay -clock [get_clocks C_CLK] -max [expr
$OutputSetupTime] [get_ports {BE[*] DATA[*]}]
set_output_delay -add_delay -clock [get_clocks C_CLK] -min [expr 0 -
$OutputHoldTime] [get_ports {WR_N}]
set_output_delay -add_delay -clock [get_clocks C_CLK] -min [expr 0 -
$OutputHoldTime] [get_ports {BE[*] DATA[*]}]

#My design can't meet normal timing, so we add a clock period of delay for the
```


Copyright © Future Technology Devices International Ltd

```
logic.
#My input comes in normally, but my output is delayed by a whole clock period.
#Since the FT601 will stop accepting output it's okay for my master to keep
trying to send data
#for another clock period.
set_multicycle_path -from [get_clocks C_CLK] -to [get_ports {WR_N}] -setup 2
set_multicycle_path -from [get_clocks C_CLK] -to [get_ports {BE[*] DATA[*]}] -
setup 2
set_multicycle_path -from [get_clocks C_CLK] -to [get_ports {WR_N}] -hold -end 2
set_multicycle_path -from [get_clocks C_CLK] -to [get_ports {BE[*] DATA[*]}] -
hold -end 2

set_multicycle_path -from [get_ports RXF_N] -to [get_clocks C_CLK] -setup 1
set_multicycle_path -from [get_ports {BE[*] DATA[*]}] -to [get_clocks C_CLK] -
setup 1
set_multicycle_path -from [get_ports RXF_N] -to [get_clocks C_CLK] -hold -end 1
set_multicycle_path -from [get_ports {BE[*] DATA[*]}] -to [get_clocks C_CLK] -
hold -end 1
```


Copyright © Future Technology Devices International Ltd

5 245 FIFO Master Example

This section goes over a 245 Synchronous FIFO master example. The Quartus project for this example can be downloaded at: FT60X Examples

5.1 M245_FIFO_Controller Interface

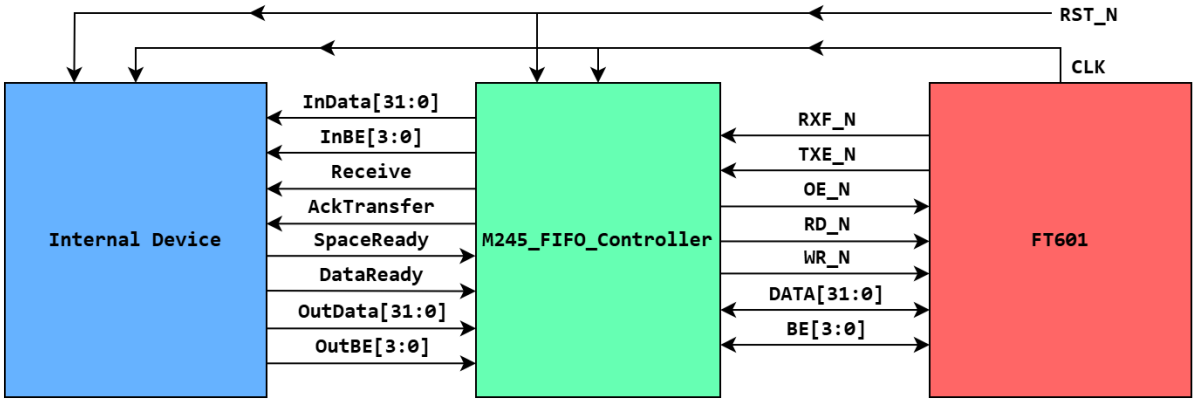


Figure 16 M245_FIFO_Controller Interface Diagram

Figure 16 shows how our example FIFO master (M245_FIFO_Controller) is connected to a FT601 and an Internal Device (some arbitrary module) that needs to send/receive data to/from the FT601. The M245_FIFO Controller in this example is essentially another data bridge. Table 13 goes over the interface between the M245_FIFO_Controller & the Internal Device.

Internal Device Interface	Internal Device Direction	Description
InData[31:0]	Input	Data for Internal Device to receive.
InBE[3:0]	Input	BE for Internal Device to receive.
Receive	Input	1'b1 = Internal Device must store incoming data. 1'b0 = Internal Device must send incoming data.
Acktransfer	Input	1'b1 = Acknowledge successful data read/write cycle. 1'b0 = No data transfer.
SpaceReady	Output	Internal Device is ready to receive data.
DataReady	Output	Internal Device is ready to send data.
OutData[31:0]	Output	Data to send to FT601.
OutBE[3:0]	Output	BE to send to FT601.

Table 13 245 Internal Device Interface

The M245_FIFO_Controller handles the 245 Synchronous FIFO protocol and scheduling read/write transactions. The Internal Device only needs to present its ability to send/receive (SpaceReady/DataReady). If the Internal Device is able to send/receive, the M245_FIFO_Controller will initiate a FIFO transaction. If the Internal Device was by chance a FIFO buffer, it would only increment/decrement its internal FIFO address on clock cycles where AckTransfer was high.

Copyright © Future Technology Devices International Ltd

5.2 M245_FIFO_Controller Operation

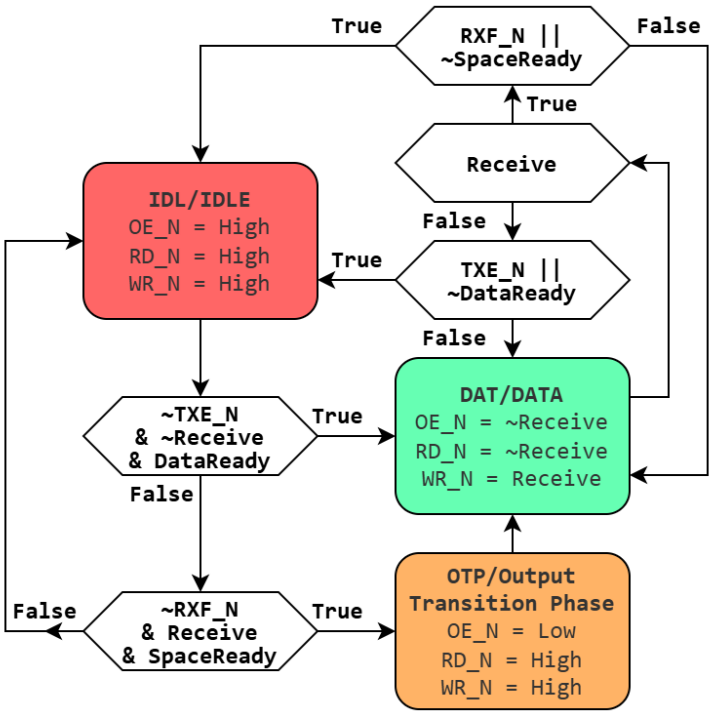


Figure 17 M245_FIFO_Controller State Machine

Figure 17 shows the state machine of the M245_FIFO_Controller. The state machine only has three states (IDL, OTP, and DAT), logic block paths between states are evaluated in one transition. When sending data to the FT601, the OTP phase is not entered. Table 14 describes all the unique values used in the state machine. When in the IDL state, the M245_FIFO_Controller checks if a specific endpoint is ready to send/receive data and changes which endpoint it checks each clock cycle.

State Machine Value	Description
Receive	If true, the Internal Device must receive data from the FT601. If false, the Internal Device must send data to the FT601.

Table 14 M245_FIFO_Controller State Machine Unique Values

5.3 245 FIFO Master Example Module Descriptions

This section goes over the example 245 FIFO Master module descriptions.

Module	Description
M245_FIFO_Master.sv	The top level module that interfaces between the FT601 and the M245_FIFO_Controller is purely for interfacing and has no logic.
M245_FIFO_Controller.sv	The M245_FIFO_Controller module that acts as a 245 FIFO protocol bridge between the FT601 and some Internal Device (module). It uses a round-robin method to check which endpoints are ready to send/receive data and will initiate transactions if the Internal device is also ready to send/receive data.
M245_FIFO_Buffer.sv	An example Internal Device module. It is a data loopback FIFO buffer with 32 KiB of total space. It will store data sent by the FT601 and send it back to the FT601 when requested.

Copyright © Future Technology Devices International Ltd

Module	Description
BYTE_RAM.sv	1-byte x 8192 word RAM IP block (BRX). M245_FIFO_Buffer uses four of these to retrieve/store values transferred on the bus. M245_FIFO_Buffer uses logic and these individual buffers so it can store/read memory at 1-byte boundaries as opposed to 4-byte boundaries. This allows M245_FIFO_Buffer to store 8191 bytes from one write pipe call and still have 1 byte of space left that can store a byte from another write pipe call. Ensuring space is never wasted in the RAM.
M245_FIFO_Counter.sv	An example Internal Device module. It is a 32-bit counter that is always ready to send/receive data. When receiving data (RXF_N = 1'b0), the 32-bit counter will be set to the last cycle of data received. When sending data (RXF_N = 1'b0), the 32-bit counter will send its current value and increment every clock cycle. If the FT601 requested 16 bytes and the counter was set to 0. The FT601 would receive four 32-bit values (0, 1, 2, & 3). If the FT601 wrote 16 bytes, the counter would be set to the value of the last four bytes. If the FT601 wrote 32'd5, and then requested 4 bytes of data, it would receive 32'd5 back and the counter would increment to 32'd6.
M245_FIFO_Video.sv	An example Internal Device module. It is a 1-channel YUV422 video data generator. It sends 1920x1080 YUV422 frames of a color-changing bouncing box to the USB host. A FT601 will receive the raw header & video data directly. A FT602 will display actual video.

Table 15 245 Master Example Module Descriptions

The below block (Figure 18) in M245_FIFO_Controller.sv can be modified to change what Internal Device example module is attached to the M245_FIFO_Controller (change M245_FIFO_Buffer to M245_FIFO_Counter & vice-versa). You can also modify it to use your own custom module.

```
M245_FIFO_Buffer M245FB(
    .CLK(CLK),
    .RST_N(RST_N),
    .Receive(Receive),
    .AckTransfer(AckTransfer),
    .InData(DATA_IN),
    .InBE(BE_IN),
    .OutData(OutDATA),
    .OutBE(OutBE),
    .SpaceReady(SpaceReady),
    .DataReady(DataReady)
);
```

Figure 18 M245_FIFO_Controller.sv Internal Device Block

Copyright © Future Technology Devices International Ltd

5.4 245 FIFO Master Example Constraints

Below are the constraints used for the 245 FIFO master example. The constraints are gone into more depth in Section 6.0.

```
#Specify the clock period.
set Period 10.000
set InputSetupTime 3.0
set InputHoldTime 3.5
set OutputSetupTime 1.0
set OutputHoldTime 4.8

create_clock -name C_CLK -period $Period [get_ports {CLK_FIFO}]

# Don't care about RST_N timing. Non-critical, Set false path.
set_false_path -from [get_ports {RST_N}] -to [get_registers *]

# Set a high clock uncertainty of 10% to increase reliability.
set_clock_uncertainty [expr $Period / 10.0] -from [get_clocks C_CLK] -to
[get_clocks C_CLK]

#Input constraints are correct.
set_input_delay -add_delay -clock [get_clocks C_CLK] -max [expr 0 -
$InputSetupTime] [get_ports {RXF_N TXE_N}]
set_input_delay -add_delay -clock [get_clocks C_CLK] -max [expr 0 -
$InputSetupTime] [get_ports {BE[*] DATA[*]}]
set_input_delay -add_delay -clock [get_clocks C_CLK] -min [expr 0 +
$InputHoldTime] [get_ports {RXF_N TXE_N}]
set_input_delay -add_delay -clock [get_clocks C_CLK] -min [expr 0 +
$InputHoldTime] [get_ports {BE[*] DATA[*]}]

#Output constraints are correct.
set_output_delay -add_delay -clock [get_clocks C_CLK] -max [expr $OutputSetupTime]
[get_ports {WR_N OE_N RD_N}]
set_output_delay -add_delay -clock [get_clocks C_CLK] -max [expr $OutputSetupTime]
[get_ports {BE[*] DATA[*]}]
set_output_delay -add_delay -clock [get_clocks C_CLK] -min [expr 0 -
$OutputHoldTime] [get_ports {WR_N OE_N RD_N}]
set_output_delay -add_delay -clock [get_clocks C_CLK] -min [expr 0 -
$OutputHoldTime] [get_ports {BE[*] DATA[*]}]

#My design can't meet normal timing, so we add a clock period of delay for the
logic.
#My input comes in normally, but my output is delayed by a whole clock period.
#Since the FT601 will stop accepting output it's okay for my master to keep trying
to send data
#for another clock period.
set_multicycle_path -from [get_clocks C_CLK] -to [get_ports {WR_N OE_N RD_N}] -
setup 2
set_multicycle_path -from [get_clocks C_CLK] -to [get_ports {BE[*] DATA[*]}] -
setup 2
set_multicycle_path -from [get_clocks C_CLK] -to [get_ports {WR_N OE_N RD_N}] -
hold -end 2
set_multicycle_path -from [get_clocks C_CLK] -to [get_ports {BE[*] DATA[*]}] -hold
-end 2

set_multicycle_path -from [get_ports {RXF_N TXE_N}] -to [get_clocks C_CLK] -setup
1
```


Copyright © Future Technology Devices International Ltd

```
set_multicycle_path -from [get_ports {BE[*] DATA[*]}] -to [get_clocks C_CLK] -
setup 1
set_multicycle_path -from [get_ports {RXF_N TXE_N}] -to [get_clocks C_CLK] -hold -
end 1
set_multicycle_path -from [get_ports {BE[*] DATA[*]}] -to [get_clocks C_CLK] -hold
-end 1
```


Copyright © Future Technology Devices International Ltd

6 Constraints Explanation

This section will explain the constraints used for the Multi-Channel FIFO master example; the same logic can be applied to the 245 Synchronous FIFO master example. Timing diagrams are pulled straight from Quartus Timing Analyzer to show the constraints working as intended. These constraints are specific to the FT60X and the example FIFO master implementations and should not be copied without understanding what they do. Different FIFO master implementations may need different constraints which should be evaluated by their implementer.

6.1 create_clock

```
create_clock -name C_CLK -period $Period [get_ports {CLK_FIFO}]
```

The create_clock command is used to create the FIFO clock used in our constraints.

6.2 set_false_path

```
# Don't care about RST_N timing. Non-critical, Set false path.
set_false_path -from [get_ports {RST_N}] -to [get_registers *]
```

The set_false_path command is used here for the RST_N pin as its timing isn't critical. This is done to prevent Quartus warning about RST_N not being constrained.

6.3 set_clock_uncertainty

```
# Set a high clock uncertainty of 10% to increase reliability.
set_clock_uncertainty [expr $Period / 10.0] -from [get_clocks C_CLK] -to
[get_clocks C_CLK]
```

The set_clock_uncertainty command is used to set the clock uncertainty to 10% of the period. This is done to increase reliability as real-world factors will affect clock skew & jitter.

6.4 set_input_delay

```
#Input constraints are correct.
set_input_delay -add_delay -clock [get_clocks C_CLK] -max [expr 0 -
$InputSetupTime] [get_ports {RXF_N}]
set_input_delay -add_delay -clock [get_clocks C_CLK] -max [expr 0 -
$InputSetupTime] [get_ports {BE[*] DATA[*]}]
set_input_delay -add_delay -clock [get_clocks C_CLK] -min [expr 0 +
$InputHoldTime] [get_ports {RXF_N}]
set_input_delay -add_delay -clock [get_clocks C_CLK] -min [expr 0 +
$InputHoldTime] [get_ports {BE[*] DATA[*]}]
```

The set_input_delay command is used to set the maximum and minimum input delay.

The maximum delay is the maximum amount of time after the clock edge the input will be valid for. The FT60X guarantees output is valid 3.0 ns ($InputSetupTime) before the clock edge. So, we set the maximum delay to -$InputSetupTime (-3.0 ns) to indicate input is always valid 3.0 ns before the clock edge. The below setup slack waveform (Figure 19) from Quartus Timing Analyzer shows this delay around the launch edge dependent on this input with the data delay (delay from logic) starting from the start of this input delay.

Copyright © Future Technology Devices International Ltd

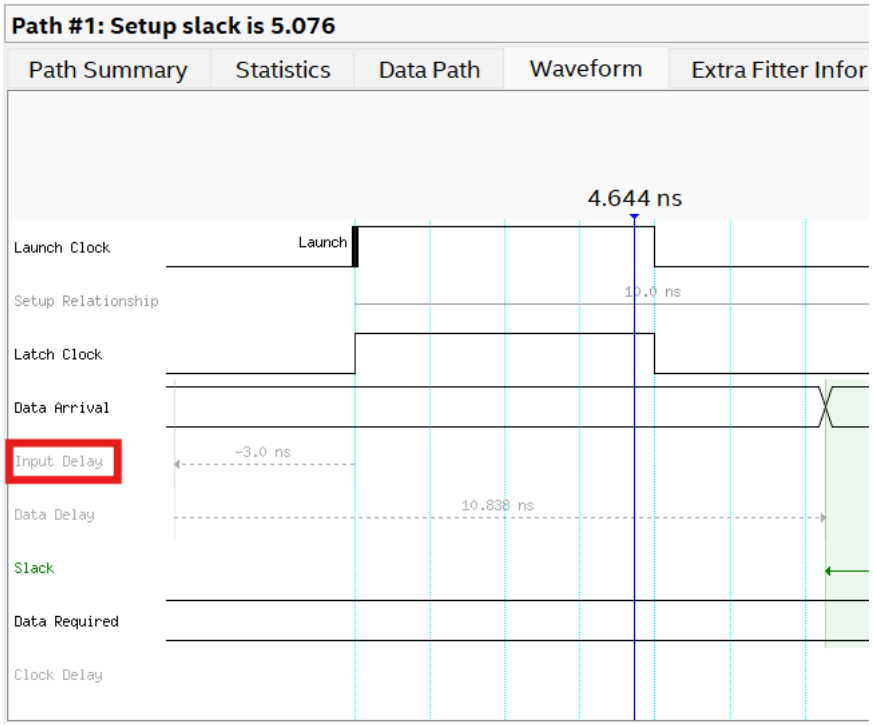


Figure 19 Input Delay Max

The minimum delay is the minimum amount of time after the clock edge the input will be valid for. The FT60X guarantees output is valid 3.5 ns ($InputHoldTime) after the clock edge. So we set the minimum delay to $InputHoldTime (3.5 ns) to indicate input is always valid up to 3.5 ns after the clock edge. The below hold slack waveform (Figure 20) from Quartus Timing Analyzer shows this delay around the launch edge dependent on this input with the data delay (delay from logic) starting from the start of this input delay.

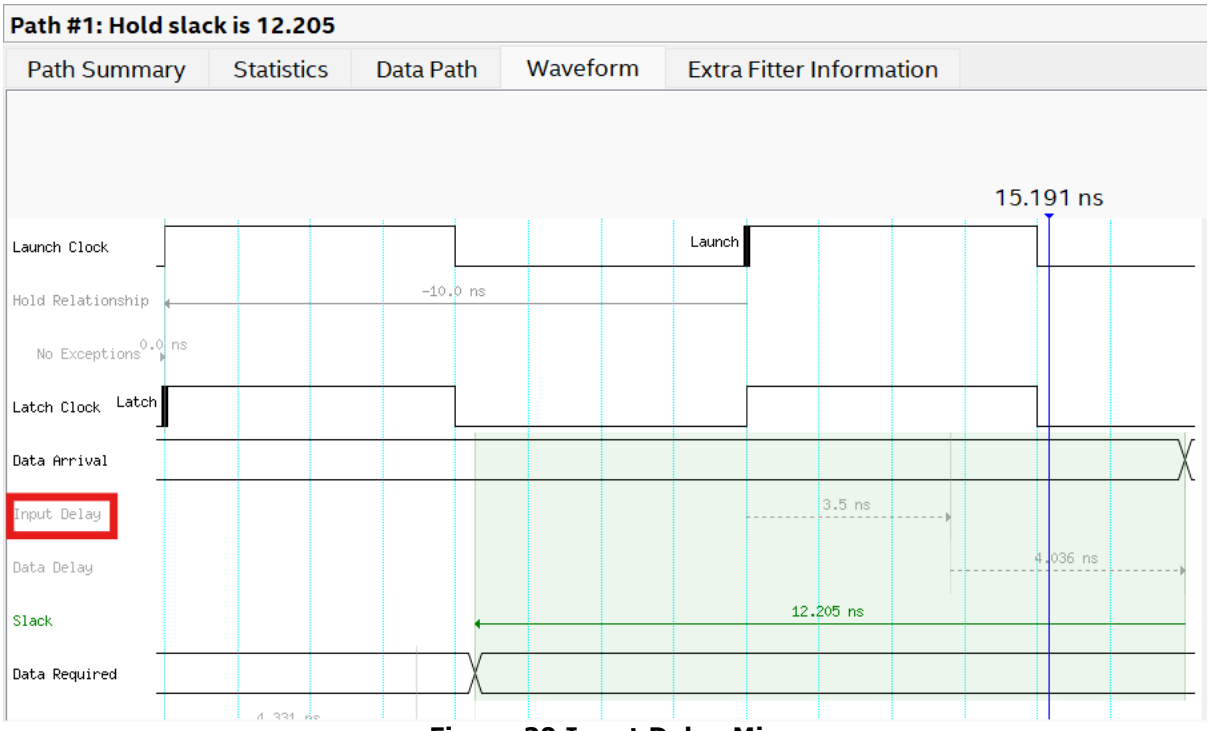


Figure 20 Input Delay Min

Copyright © Future Technology Devices International Ltd

6.5 set_output_delay

```
#Output constraints are correct.
set_output_delay -add_delay -clock [get_clocks C_CLK] -max [expr
$OutputSetupTime] [get_ports {WR_N}]
set_output_delay -add_delay -clock [get_clocks C_CLK] -max [expr
$OutputSetupTime] [get_ports {BE[*] DATA[*]}]
set_output_delay -add_delay -clock [get_clocks C_CLK] -min [expr 0 -
$OutputHoldTime] [get_ports {WR_N}]
set_output_delay -add_delay -clock [get_clocks C_CLK] -min [expr 0 -
$OutputHoldTime] [get_ports {BE[*] DATA[*]}]
```

The set_output_delay command is used to set the maximum and minimum output delay.

The maximum delay is the maximum amount of time before the clock edge the output must be valid for. The FT60X requires input to be valid 1.0 ns ($OutputSetupTime) before the clock edge. So we set the maximum delay to $OutputSetupTime (1.0 ns) to indicate output is always valid 1.0 ns before the clock edge. The below setup slack waveform (Figure 21) from Quartus Timing Analyzer shows this delay (plus our clock uncertainty) on this output before the latch edge dependent on it.

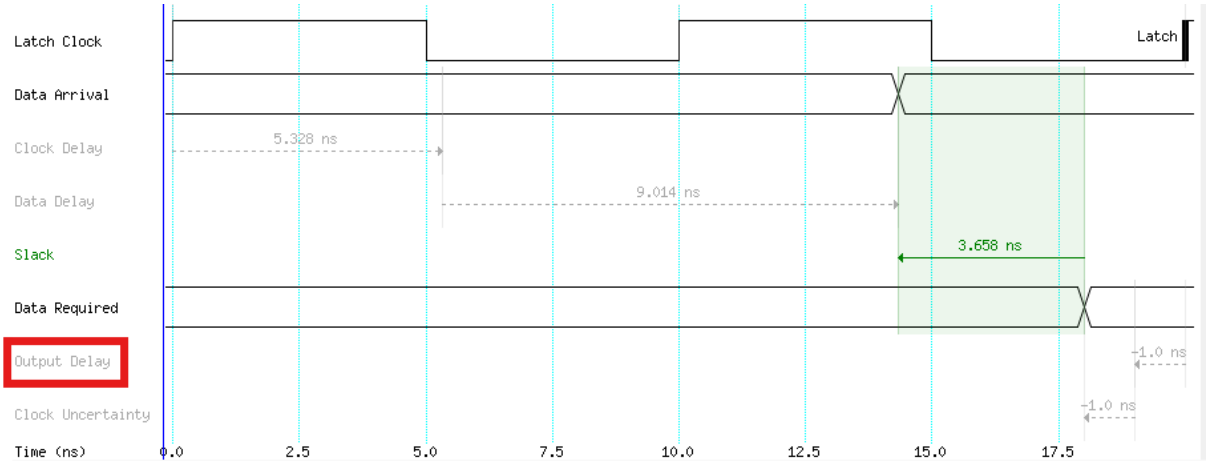


Figure 21 Output Delay Max

The minimum delay is the minimum amount of time before the clock edge the output must be valid for. The FT60X requires input to be valid up to 4.8 ns ($OutputHoldTime) after the clock edge. So we set the minimum delay to -$OutputHoldTime (-4.8 ns) to indicate output is always valid up to 4.8 ns after the clock edge. The below hold slack waveform (Figure 22) from Quartus Timing Analyzer shows this delay (plus our clock uncertainty) on this output after the latch edge dependent on it.

Copyright © Future Technology Devices International Ltd

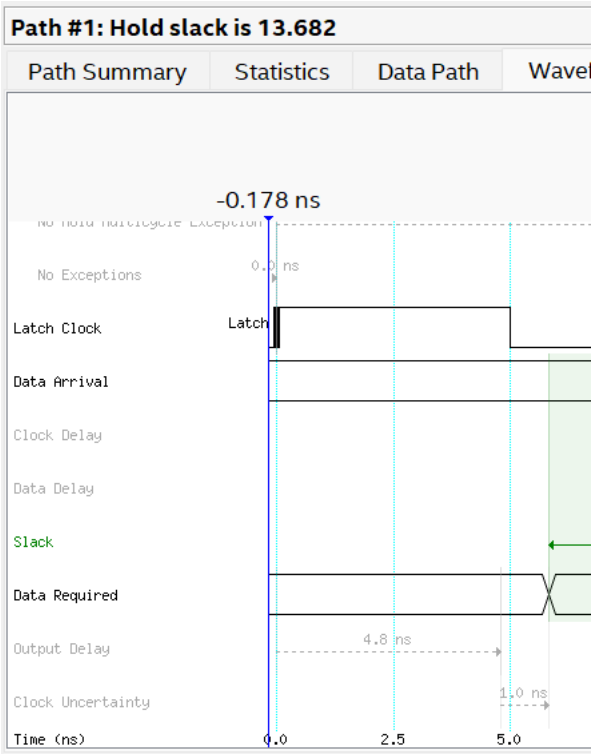


Figure 22 Output Delay Min

6.6 set_multicycle_path

```
#My design can't meet normal timing, so we add a clock period of delay for the
logic.
#My input comes in normally, but my output is delayed by a whole clock period.
#Since the FT601 will stop accepting output it's okay for my master to keep
trying to send data
#for another clock period.
set_multicycle_path -from [get_clocks C_CLK] -to [get_ports {WR_N}] -setup 2
set_multicycle_path -from [get_clocks C_CLK] -to [get_ports {BE[*] DATA[*]}] -
setup 2
set_multicycle_path -from [get_clocks C_CLK] -to [get_ports {WR_N}] -hold -end 2
set_multicycle_path -from [get_clocks C_CLK] -to [get_ports {BE[*] DATA[*]}] -
hold -end 2
```

The set_multicycle_path command is used to set the number of cycles the data path must have for setup & hold checks.

The FT60X examples in this guide cannot meet normal setup & hold times within a single clock cycle. However, if we delay output by a whole clock period (two cycles instead of one), our design can meet timing. Do note the FIFO master examples still clock in input normally and transactions are completely controlled by the FIFO master. This means we can delay/shift the entire transaction by one clock cycle so our design can meet timing. This shift in timing is applied to ALL of our outputs which control transactions, meaning transactions will occur the same but when they start will be delayed by a clock cycle.

The below setup slack waveform (Figure 23) from Quartus Timing Analyzer shows how the expected latch edge for our output is one clock cycle further (2 cycles) from the launch edge as opposed to the normal zero clock cycles further (1 cycle). Our output still changes every clock cycle, but its expected latch edge is a cycle further than normal. You may have already noticed this in Figure 21 previously.

Copyright © Future Technology Devices International Ltd

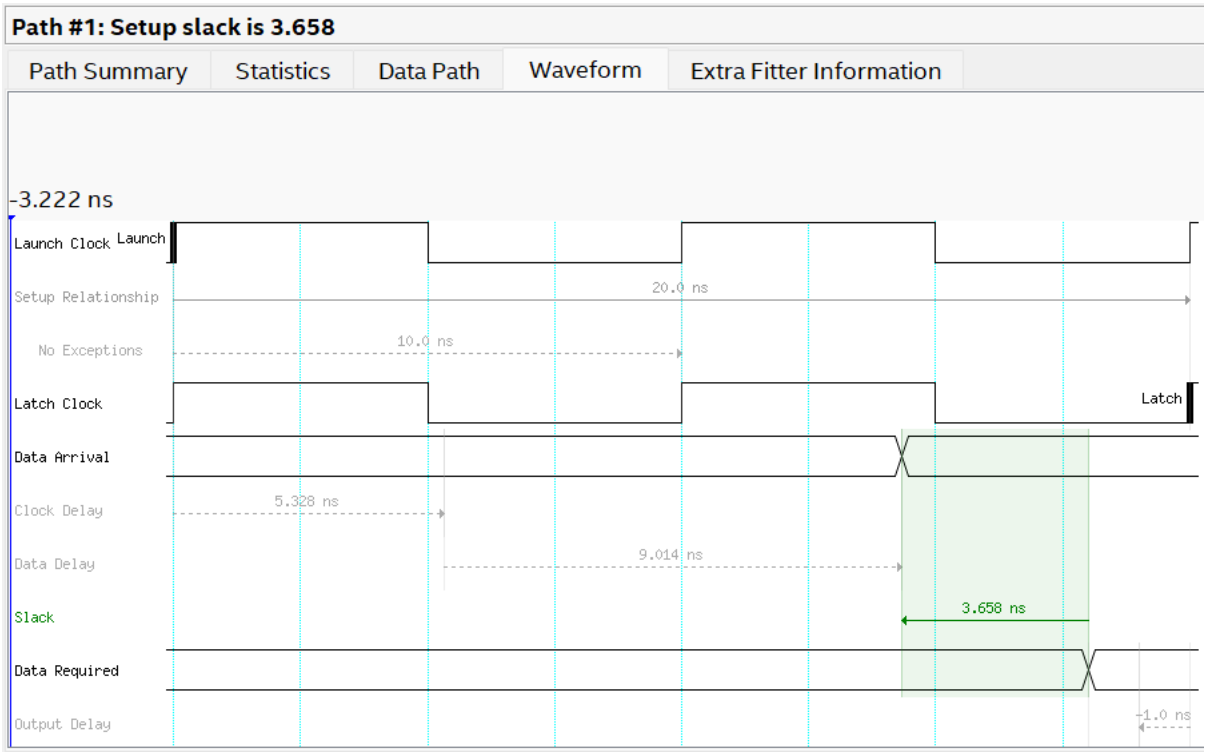


Figure 23 2-Cycle Multicycle Path

Copyright © Future Technology Devices International Ltd

7 Conclusion

All information used to make this document and software example was derived from:

FT600Q-FT601Q SuperSpeed USB3.0 IC Datasheet

AN_370 FT60X Configuration Programmer User Guide

AN_379 D3xx Programmers Guide

AN_386 FTDI FT600 Maximize Performance

AN_412 FT600/FT601 Bridge Chips Integration

AN_421 FIFO Bus Master For FT60x

This Technical Note is not comprehensive of all aspects of FTDI's FT60X chips but covers the basics needed for successful data transfer alongside FIFO master examples. We recommend reviewing the datasheet and other ANs for more details or topics not covered here.

Copyright © Future Technology Devices International Ltd

8 Contact Information

For contact information where you can find contact details for FTDI offices, please visit the Technical Support page on FTDI website.

For distributer and sales representatives contact details in your country, please visit the Sales Network page on FTDI website.

System and equipment manufacturers and designers are responsible to ensure that their systems, and any Future Technology Devices International Ltd (FTDI) devices incorporated in their systems, meet all applicable safety, regulatory and system-level performance requirements. All application-related information in this document (including application descriptions, suggested FTDI devices and other materials) is provided for reference only. While FTDI has taken care to assure it is accurate, this information is subject to customer confirmation, and FTDI disclaims all liability for system designs and for any applications assistance provided by FTDI. Use of FTDI devices in life support and/or safety applications is entirely at the user's risk, and the user agrees to defend, indemnify, and hold harmless FTDI from any and all damages, claims, suits, or expense resulting from such use. This document is subject to change without notice. No freedom to use patents or other intellectual property rights is implied by the publication of this document. Neither the whole nor any part of the information contained in, or the product described in this document, may be adapted, or reproduced in any material or electronic form without the prior written consent of the copyright holder. Future Technology Devices International Ltd, Unit 1, 2 Seaward Place, Centurion Business Park, Glasgow G41 1HH, United Kingdom. Scotland Registered Company Number: SC136640

Copyright © Future Technology Devices International Ltd

Appendix A – References

Document / Web References

List of relevant FTDI and external datasheets, application notes, website links and other supporting documents for reference.

IC Datasheets

- FT600Q-FT601Q SuperSpeed USB3.0 IC Datasheet

Application Notes

- AN_370 FT60X Configuration Programmer User Guide
- AN_379 D3xx Programmers Guide
- AN_386 FTDI FT600 Maximize Performance
- AN_412 FT600/FT601 Bridge Chips Integration
- AN_421 FIFO Bus Master For FT60x

For information about FTDI product categories, please visit the Products page on FTDI website.

For information about other relevant documents, please visit the Documents page on FTDI website.

Acronyms and Abbreviations

Terms	Description
245	245 Synchronous (for FT60X FIFO protocol mode)
D3XX	FTDI's "Direct" driver for SuperSpeed (FT60X) devices
FIFO	First In First Out
FT60X	FTDI's FT600 or FT601 devices.
MC	Multi-Channel (for FT60X FIFO protocol mode)
USB	Universal Serial Bus

Copyright © Future Technology Devices International Ltd

Appendix B – List of Tables and Figures

List of Tables

Table 1 FT60X Interfaces, Channels, and Endpoints 5
Table 2 FIFO Size Per Channel Configuration 5
Table 3 MaxPacketSize Values 8
Table 4 FIFO Bus Output Timing 9
Table 5 MC FIFO Bus Phases 10
Table 6 MC Command Phase Values 10
Table 7 MC FIFO Status Bits 10
Table 8 Data Phase Byte Enable Values 11
Table 9 245 FIFO Bus Phases 13
Table 10 MC Internal Device Interface 17
Table 11 MC_FIFO_Controller State Machine Unique Values 18
Table 12 MC Master Example Module Descriptions 19
Table 13 245 Internal Device Interface 22
Table 14 M245_FIFO_Controller State Machine Unique Values 23
Table 15 245 Master Example Module Descriptions 24

List of Figures

Figure 1 FT60X 4-Channel Normal Session 6
Figure 2 FT60X 4-Channel Session Underrun 7
Figure 3 FT601 4-Channel Ignore Bus-Width Underruns Session 8
Figure 4 FIFO Bus Input/Output Timing Diagram 9
Figure 5 MC FIFO Master Full Read Transaction 11
Figure 6 MC FIFO Master Partial Read Transaction 12
Figure 7 MC FIFO Master Partial Write Transaction 12
Figure 8 MC FIFO Master Full Write Transaction 13
Figure 9 245 FIFO Master Full Read Transaction 14
Figure 10 245 FIFO Master Partial Read Transaction 15
Figure 11 245 FIFO Master Partial Write Transaction 15
Figure 12 245 FIFO Master Full Write Transaction 16
Figure 13 MC_FIFO_Controller Interface Diagram 17
Figure 14 MC_FIFO_Controller State Machine 18
Figure 15 MC_FIFO_Controller.sv Internal Device Block 20
Figure 16 M245_FIFO_Controller Interface Diagram 22
Figure 17 M245_FIFO_Controller State Machine 23
Figure 18 M245_FIFO_Controller.sv Internal Device Block 24
Figure 19 Input Delay Max 28

Copyright © Future Technology Devices International Ltd

Figure 20 Input Delay Min 28
Figure 21 Output Delay Max 29
Figure 22 Output Delay Min 30
Figure 23 2-Cycle Multicycle Path 31

Copyright © Future Technology Devices International Ltd

Appendix C – Revision History

Document Title: TN_191 FT60X Data Transfer & FIFO Master Basics

Document Reference No.: FT_001666

Clearance No.: FTDI#629

Document Feedback: Send Feedback

Revision	Changes	Date
1.0	Initial Release	08-07-2026

Copyright © Future Technology Devices International Ltd